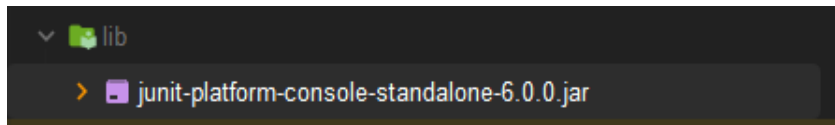


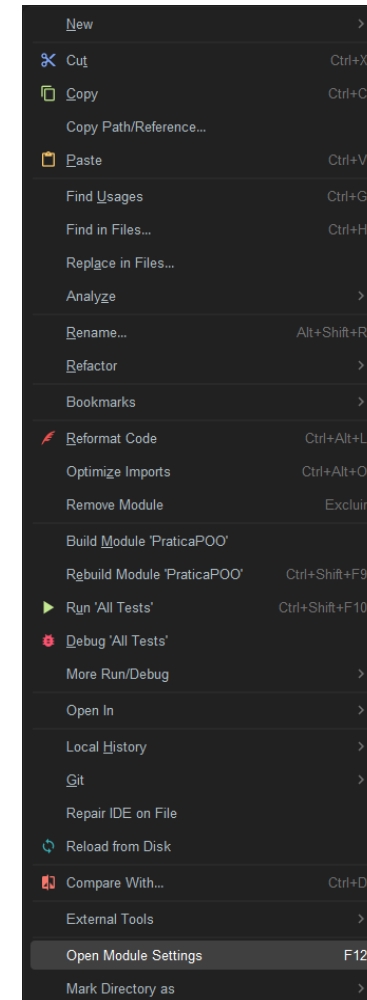
Testes de unidade com JUnit

Como testar com JUnit no IntelliJ

1. Recomenda-se criar uma pasta lib dentro do projeto, e adicionar o .jar do JUnit dentro dessa pasta

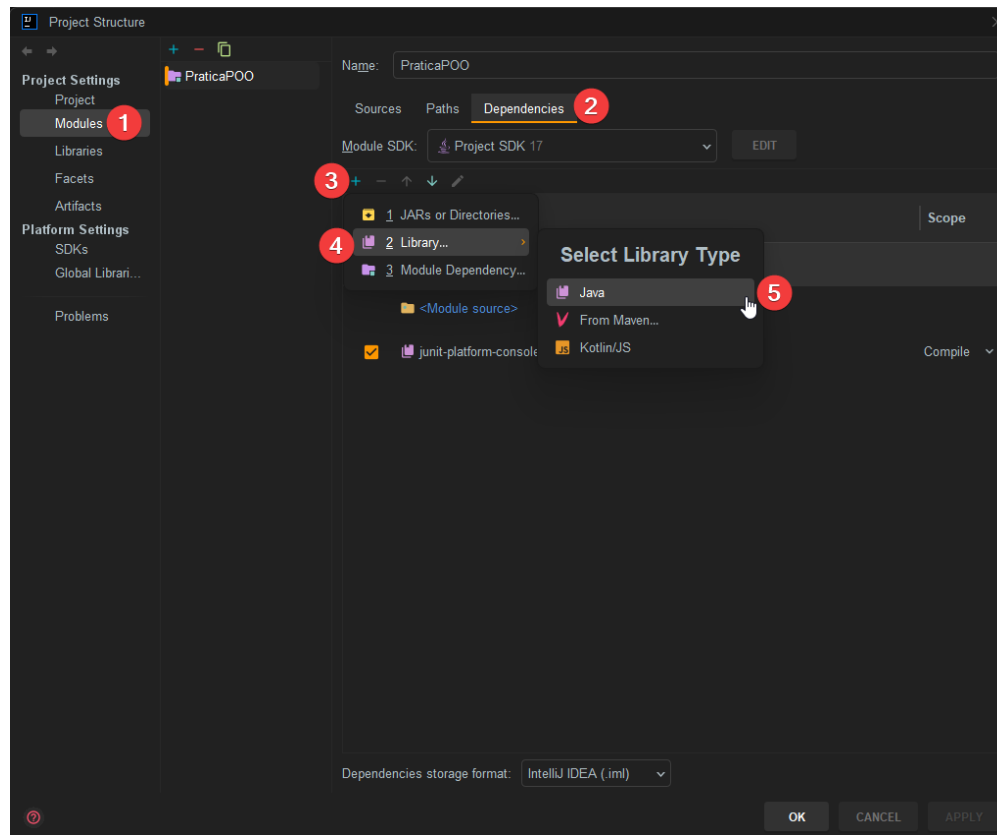


2. Adicionar essa biblioteca como um módulo do projeto.
Clicando com o botão direito do mouse sobre o projeto e acessando a opção: "Open Module Settings" conforme a imagem ao lado



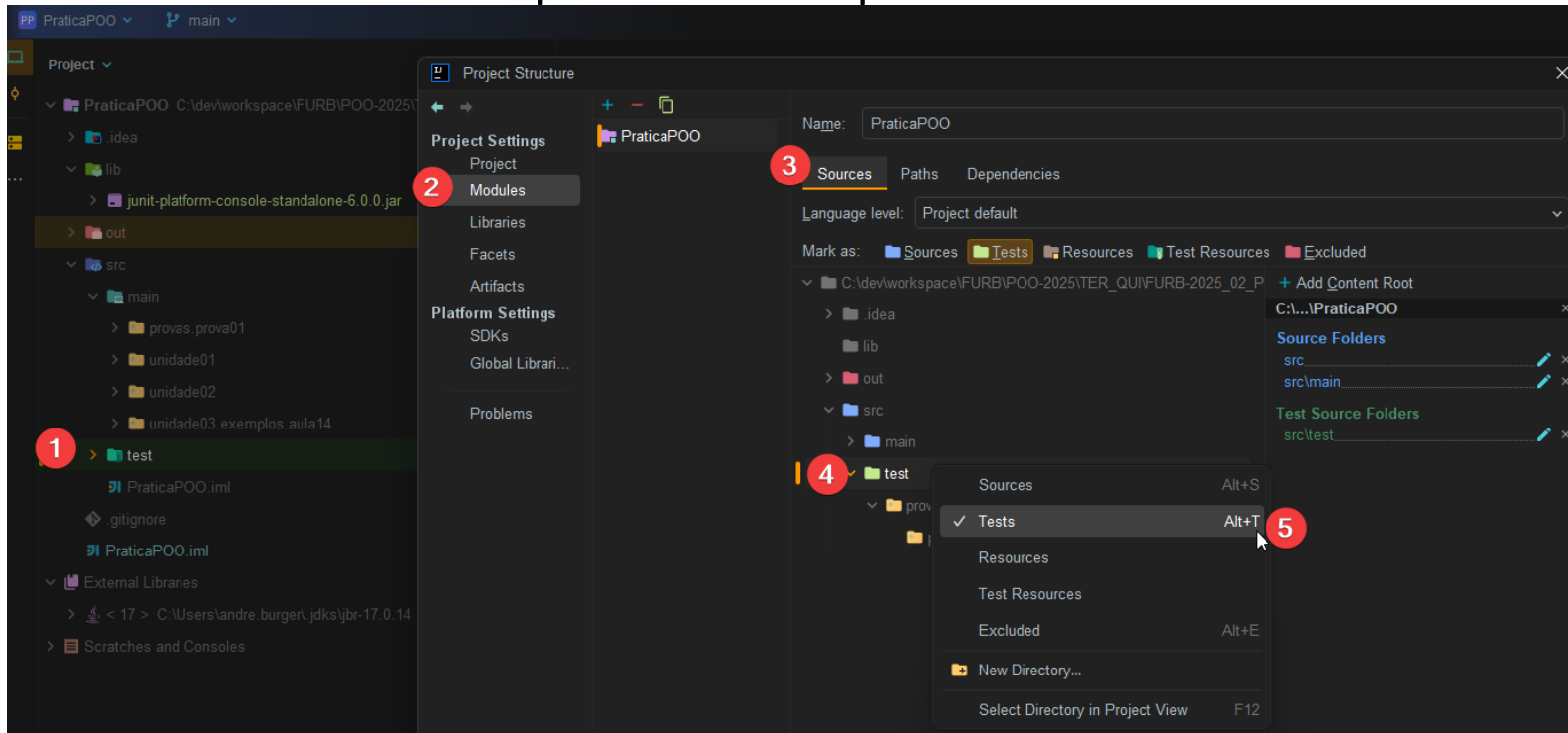
Como testar com JUnit no IntelliJ

- Seguir o passo a passo conforme a imagem abaixo, e ao final selecionar o arquivo que foi adicionado na pasta lib.



Como testar com JUnit no IntelliJ

1. Na pasta src, criar uma nova pasta chamada test
2. Acessar novamente a opção: "Open Module Settings" conforme a imagem abaixo e definir que essa será a pasta dos testes



Como testar com JUnit em outras IDEs

1. Deve seguir os mesmos procedimentos realizados no IntelliJ, seguindo o padrão de cada IDE, mas o passo a passo é o mesmo.
 - Colocar o jar em uma pasta do projeto
 - Adicionar esse jar como uma biblioteca do projeto
 - Criar a pasta onde ficarão as classes de teste (test)
 - Depende de cada IDE se precisa definir essa pasta como uma pasta de teste ou não.

Testes com JUnit

- Como vamos implementar testes para a camada de negócio, haverá uma classe “paralela” (de teste) para cada classe a ser testada
- Em geral, o nome desta classe é igual ao nome da classe a ser testada, porém com sufixo “Test”
- Para cada “caso de teste”, criar um método e introduzir a anotação @Test
 - O método deve ser público e não pode ter parâmetros
 - O método não pode retornar dados (void)
 - Dentro deste método, deve-se utilizar um comando *assert*, para validar uma situação

Como testar com JUnit

- Exemplo:

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import static org.junit.jupiter.api.*;
3
4 public class CalculadoraTest {
5
6     @Test
7     public void test1_SomarNumeros() {
8         Calculadora calc = new Calculadora();
9         int resultado = calc.somar(10,75);
10        assertEquals(85, resultado);
11    }
12
13 }
```

Valor esperado

Valor obtido

“Espero que o somar()
resulte em 85”

Métodos Assert

- Os métodos *assert* são utilizados para verificar se uma condição é verdadeira. Caso a verificação não seja bem sucedida, é lançada uma exceção

Método	Descrição
<code>assertEquals(long esperado, long real)</code>	Verifica se os dois números inteiros são iguais
<code>assertEquals(double esperado, double real, double limite)</code>	Verifica se dois números decimais são iguais, sem atingir o limite de diferença
<code>assertEquals(objetoEsperado, objetoReal)</code>	Verifica se dois objetos são iguais
<code>assertTrue(boolean condição)</code>	Verifica se a condição é verdadeira
<code>assertFalse(boolean condição)</code>	Verifica se a condição é falsa
<code>assertNotNull(objeto)</code>	Verifica se a variável referencia um objeto
<code>assertNull(objeto)</code>	Verifica se a variável não referencia um objeto

Anotações JUnit

Anotação	Descrição / Funcionalidade	Observações
<code>@Test</code>	Marca um método como um teste unitário.	É o ponto de entrada principal para um caso de teste.
<code>@BeforeEach</code>	Executa antes de cada método de teste.	Ideal para inicializar variáveis ou objetos comuns a todos os testes.
<code>@AfterEach</code>	Executa depois de cada método de teste.	Usado para limpar recursos (ex: fechar conexões, resetar dados).
<code>@BeforeAll</code>	Executa uma vez antes de todos os testes da classe.	O método deve ser <code>static</code> (ou em instância se a classe for anotada com <code>@TestInstance(Lifecycle.PER_CLASS)</code>).
<code>@AfterAll</code>	Executa uma vez depois de todos os testes da classe.	Também deve ser <code>static</code> (mesma regra do <code>@BeforeAll</code>).
<code>@DisplayName</code>	Define um nome legível para o teste ou classe de teste.	Facilita a leitura dos relatórios de execução.

Anotações JUnit

`@Disabled`

Desabilita temporariamente um teste ou classe de testes.

Útil para pular testes em desenvolvimento ou que dependem de algo externo.

`@Tag`

Agrupa testes com uma determinada tag.

Permite executar subconjuntos de testes (ex: `@Tag("integration")`).

`@Timeout`

Define um **tempo máximo de execução** para o teste.

Se excedido, o teste falha automaticamente.

`@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`

Define que a **ordem de execução dos testes** será determinada pela anotação `@Order` .

É aplicada **na classe de teste**.

`@Order(1)`

Define a **ordem de execução** de um método de teste.

Funciona apenas se a classe estiver anotada com `@TestMethodOrder` .

Casos de teste com o mesmo contexto

- Quando dois ou mais casos de teste compartilham o mesmo contexto, é possível definir o contexto num método que contém a anotação *@BeforeAll*

```
@BeforeAll  
public void inicializarContexto() {  
    calc = new Calculadora();  
}
```

- Igualmente, é possível definir um método para ser executado após cada caso de teste, com a anotação *@AfterEach*

```
@AfterEach  
public void finalizarContexto() {  
    arquivo.close();  
}
```

Exceções

- Podemos realizar testes que esperam como resultado uma exceção, como por exemplo

```
1 import static org.junit.jupiter.api.Assertions.*;
2 import static org.junit.jupiter.api.*;
3
4 public class CalculadoraTest {
5
6     @Test
7     public void test1_SomarNumeros() {
8         Calculadora calc = new Calculadora();
10         assertThrows(ArithmeticException.class, calc.dividir(5, 0));
11     }
12
13 }
```